

## Assignment 1 226

(Execution time)

| Threads | TTAS avg time (ms) | CLH avg time (ms) | MCS avg time (ms) |
|---------|--------------------|-------------------|-------------------|
| 2       | 1,972              | 4,619             | 2,885             |
| 4       | 4,426              | 7,872             | 8,921             |
| 8       | 11,953             | 598,789           | 1204,820          |
| 16      | 48,891             | 10,289            | 7871,748          |

**TTAS:** at low contention it's fine (read-only spinning on a cached value), but as threads increase, every successful acquisition invalidates other threads' cache lines → the spinning threads' test keeps hammering the shared variable → performance degrades sharply, and access can be unfair (a thread can be starved).

**CLH:** each thread spins on its own predecessor's node (local memory), so acquiring releases less cache traffic. But it's not ideal on cacheless NUMA systems, and unlock touches the predecessor's node.

**MCS:** like CLH but spins on the thread's own node, avoiding NUMA issues; FIFO-fair; slightly more complex due to the tail update.

Fairness/distribution: look at "bids won per bidder" — TTAS will likely show skewed distributions at high thread counts, while CLH/MCS stay near-even. This directly ties to your required measurement.

**Sanity check:** confirm total bids and that the final highest bid is consistent — this validates mutual exclusion and completes the workload verification.